

Sadhak Mobile Application - Development Plan

Developer: Jil Mehta **Purpose:** Technical overview and planned development actions
Status: Planning Phase - Ready to Begin

Overview

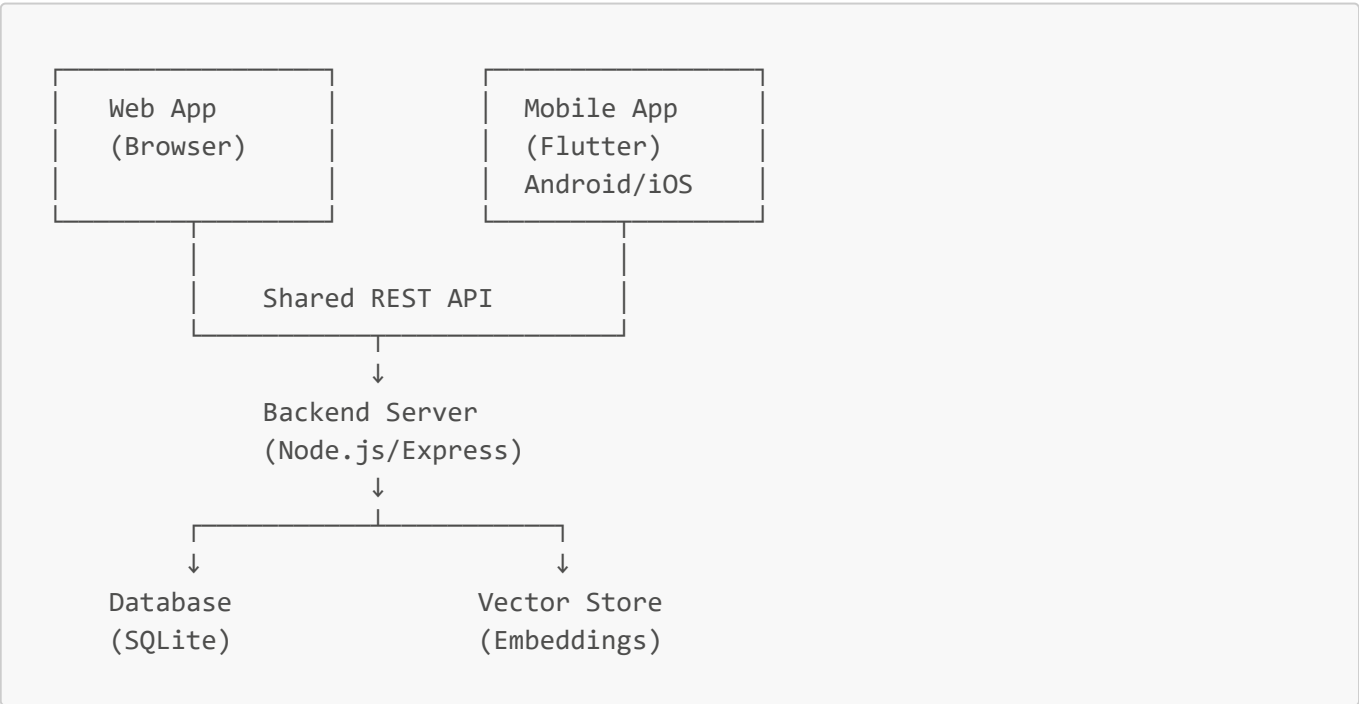
This document outlines my plan to develop a native Android mobile application for Sadhak that will share the existing backend infrastructure with the current web application. This is for informational purposes - no action required from you, just reviewing my planned approach.

Architecture

Current System



Proposed Multi-Platform Architecture



Key Point: The mobile app will consume the exact same API endpoints as the web application. No backend changes required.

Technology Stack

Mobile Application

- **Framework:** Flutter (Dart)
- **Why Flutter:**
 - Single codebase for Android + iOS
 - Native performance
 - Rich UI components out-of-the-box
 - Hot reload for rapid development
 - Strong typing with Dart

Backend (No Changes)

- **Server:** Node.js + Express
- **Database:** SQLite
- **Vector Store:** In-memory embeddings for semantic search
- **Authentication:** Google OAuth 2.0
- **API Style:** RESTful endpoints

API Endpoints to be Consumed

- **POST /auth/google** - Authentication
 - **GET /auth/user** - User session
 - **POST /search** - Semantic search
 - **POST /chat** - AI chat with RAG
 - **GET /history** - Chat history
-

Development Phases & Actions

Phase 1: Environment Setup

Timeline: 1-2 days

My Actions:

- Install Flutter SDK and configure PATH
- Set up Android Studio with Flutter/Dart plugins
- Configure Android emulator for testing
- Run **flutter doctor** to verify environment
- Create initial Flutter project: **flutter create sadhak_mobile**

Deliverable: Working development environment

Phase 2: Project Foundation

Timeline: 2-3 days

My Actions:

- Configure `pubspec.yaml` with dependencies (http, provider, google_sign_in)
- Create project folder structure (services, models, screens, widgets)
- Build API service layer with base HTTP client
- Implement environment config for dev/prod API URLs
- Set up Provider for state management
- Create app navigation structure

Deliverable: App scaffold with API integration layer

Phase 3: Authentication

Timeline: 2-3 days

My Actions:

- Create login screen UI
- Integrate Google Sign-In SDK for Flutter
- Implement authentication service calling `/auth/google`
- Add session persistence using SharedPreferences
- Create auth state provider
- Implement protected route guards

Deliverable: Working login/logout with session management

Phase 4: Search Feature

Timeline: 3-4 days

My Actions:

- Design search screen UI (input field, language toggle)
- Implement search service calling `POST /search`
- Create search result card widgets
- Add image loading for resource thumbnails
- Implement language switching (English/Hindi)
- Add loading states and error handling
- Create resource detail view

Deliverable: Fully functional search interface

Phase 5: AI Chat Feature

Timeline: 3-4 days

My Actions:

- Design chat screen UI
- Create chat bubble widgets (user/AI messages)

- Implement chat service calling `POST /chat`
- Add chat history retrieval from `GET /history`
- Implement message input with send button
- Add auto-scroll to latest message
- Handle streaming/loading states

Deliverable: Working AI chat with history

Phase 6: UI/UX Polish

Timeline: 2-3 days

My Actions:

- Apply consistent theming (colors, fonts, spacing)
- Implement responsive layouts for various screen sizes
- Add smooth transitions and animations
- Optimize image loading and caching
- Implement pull-to-refresh where applicable
- Add accessibility labels

Deliverable: Polished, production-ready UI

Phase 7: Testing & QA

Timeline: 2-3 days

My Actions:

- Write unit tests for services and models
- Test API integration with live backend
- Test on multiple Android versions (8.0+)
- Test on different screen sizes (phone/tablet)
- Test network error scenarios
- Verify offline behavior
- Performance profiling and optimization

Deliverable: Tested, stable application

Phase 8: Build & Documentation

Timeline: 1-2 days

My Actions:

- Generate release APK: `flutter build apk --release`
- Configure app signing for Play Store
- Create user documentation
- Document deployment process
- Prepare app store assets (if needed)

Deliverable: Release-ready APK

Timeline Summary

Phase	Duration	Cumulative
1. Environment Setup	1-2 days	1-2 days
2. Project Foundation	2-3 days	3-5 days
3. Authentication	2-3 days	5-8 days
4. Search Feature	3-4 days	8-12 days
5. AI Chat Feature	3-4 days	11-16 days
6. UI/UX Polish	2-3 days	13-19 days
7. Testing & QA	2-3 days	15-22 days
8. Build & Docs	1-2 days	16-24 days

Total Estimated Time: 16-24 working days (~3-5 weeks)

Note: Phases 4 and 5 can potentially overlap if needed.

Resource Requirements

Developer Time

- **Full Development:** 16-24 days (single developer)
- **Part-time (50%):** 32-48 days (~6-10 weeks)
- **Includes:** Development, testing, documentation

Testing Resources

- Android emulator (included with Android Studio)
- 2-3 physical Android devices for real-world testing (various screen sizes recommended)

Infrastructure

- **Backend:** Existing infrastructure (no changes)
 - **Hosting:** Current setup (no additional cost)
 - **Third-party Services:** Google OAuth (already configured)
-

Risk Assessment & Mitigation

Technical Risks

Risk 1: Flutter Learning Curve

- **Impact:** Medium - May slow initial development
- **Probability:** Low - Extensive documentation available
- **Mitigation:**
 - Flutter has excellent official documentation
 - Large community for troubleshooting
 - Similar concepts to React (if familiar)

Risk 2: API Compatibility Issues

- **Impact:** Low - Minor adjustments may be needed
- **Probability:** Very Low - API already tested with web app
- **Mitigation:**
 - Backend API is already stable and proven
 - Can test API calls early in Phase 2
 - No breaking changes planned for backend

Risk 3: Android Device Fragmentation

- **Impact:** Medium - UI may need adjustments for different screens
- **Probability:** Medium - Android has many device types
- **Mitigation:**
 - Flutter handles most fragmentation automatically
 - Test on multiple screen sizes early
 - Use responsive design patterns from start

Risk 4: Performance on Older Devices

- **Impact:** Medium - App may be slow on Android 8-9 devices
- **Probability:** Low - Flutter is well-optimized
- **Mitigation:**
 - Target Android 8.0+ (covers 95%+ of devices)
 - Profile performance early
 - Optimize image loading and caching

Risk 5: OAuth Flow on Mobile

- **Impact:** Medium - Authentication may need mobile-specific handling
- **Probability:** Low - Standard OAuth flow
- **Mitigation:**
 - Use official google_sign_in Flutter package
 - Test early in Phase 3
 - Fallback to web-based OAuth if needed

Project Risks

Risk 6: Scope Creep

- **Impact:** High - Could extend timeline significantly
- **Probability:** Medium - Common in development
- **Mitigation:**

- Strict feature parity with web app (no new features)
- Phase-based approach allows for scope control
- Document any additional requests separately

Risk 7: Backend API Changes

- **Impact:** High - Would require rework
 - **Probability:** Very Low - API is stable
 - **Mitigation:**
 - No planned backend changes
 - Mobile app uses existing, tested endpoints
 - Coordinate any backend changes in advance
-

Success Criteria

Functional Requirements

- ☒ User can log in with Google account
- ☒ User can search resources in English and Hindi
- ☒ User can chat with AI and receive context-aware responses
- ☒ User can view chat history
- ☒ App maintains session across restarts
- ☒ All features match web application functionality

Technical Requirements

- ☒ App runs on Android 8.0+ devices
- ☒ API calls complete within acceptable time (<3s)
- ☒ No crashes or critical bugs
- ☒ Proper error handling for network issues
- ☒ Secure token storage

Quality Requirements

- ☒ Intuitive, consistent UI
 - ☒ Smooth animations and transitions
 - ☒ Responsive design for different screen sizes
 - ☒ Follows Material Design guidelines
-

Next Immediate Actions

This Week

1. Install Flutter SDK and verify with `flutter doctor`
2. Create Flutter project structure
3. Set up API integration layer
4. Begin authentication implementation

Next Week

1. Complete authentication flow
2. Start search feature development
3. Initial testing on emulator

Following Weeks

1. Complete search and chat features
 2. UI/UX polish
 3. Comprehensive testing
 4. Build release APK
-

Future Considerations

iOS Version

- Once Android app is stable, can extend to iOS with ~70% code reuse
- Estimated additional time: 2-3 weeks
- No additional backend work required

Potential Enhancements (Post-MVP)

- Push notifications for chat responses
 - Offline mode with local caching
 - Firebase Analytics integration
 - In-app resource bookmarking
-

Summary

I will be developing a Flutter-based Android mobile application that leverages our existing backend infrastructure. The approach is low-risk, with a clear phased plan and realistic timeline of 3-5 weeks. No backend modifications are required, and the same API that powers the web app will power the mobile app.

Key Advantages:

- Minimal risk (proven tech stack, existing backend)
- No additional infrastructure costs
- Foundation for future iOS app
- Feature parity with web application

This document outlines my planned approach - no action needed from you, just keeping you informed of the development plan and timeline.

Questions or Concerns?

Feel free to reach out if you have any questions about the approach or timeline.

Prepared by: [Your Name]
Date: December 25, 2024
Version: 1.0